

zk Payment Channels: a Definition, and the First Machine-Checked Unlinkability Results for a Credit Construction

dmarz*

July 2026 — draft

Abstract

A zk payment channel is a two-party channel in which the payee learns nothing about payer identity across spends — unlinkability within the channel population — while retaining balance security and double-spend resistance. The object was named by BOLT in 2016, after which the line went dormant: zkChannels stayed a draft, its implementation was archived in 2023, and no dedicated literature formed. Meanwhile the same shape has been reinvented at least twice, as keyed-verification credit tokens (ACT, ARC) and as ZK API Usage Credits. This paper defines the object as a tuple of algorithms (Setup, Open, Spend, Redeem, Close, Dispute) with seven security games, places it against the modern lineage, and reports a Lean 4 formalization of a flat-ticket RLN credit instantiation. Spend unlinkability is machine-checked: against an adversarial payee holding BOLT §1.4’s abort/evict powers, the advantage of distinguishing which of two members emitted a whole epoch session is exactly zero — to our knowledge the first machine-checked unlinkability result for any payment-channel or credit construction. No-overspend, both balance-security theorems, closure liveness, a priced-divergence bound for an eventually-consistent multi-gateway deployment, the exculpability (framing) bound (under a stated random-oracle good-event hypothesis, with a kernel-checked query-budget composition endpoint and the final handler-coupling step still open), and the refund variant’s safety and conservation theorems — now including the full failed-upgrade cascade and finite-fleet aggregation — are machine-checked with zero **sorry** and no axiom beyond Lean’s three standard ones. The model-to-real bridges are no longer stated obligations: proof-bearing wire instances (a masked-proof encoding, an interactive Sigma protocol, and a lazy-random-oracle Fiat–Shamir compilation with explicit programming and fork-collision bounds) each discharge the zero-knowledge bridge with zero loss; an executable ledger refines every relational transition; a multi-recipient portable-deposit accounting layer with a threshold-issuance reference construction closes the definitional half of the named open problem; and one-trace composition theorems deliver the channel and wire guarantee bundles simultaneously on a single reachable trace. The definition went through eleven rounds of independent adversarial review, producing concrete counterexamples against ten successive revisions; the repairs those counterexamples forced — gateway-bound messages, close-time netting, verifiable spend counts at close — are, we argue, design requirements for any construction of this shape, and we present them as such.

1 Introduction

BOLT [21] constructed anonymous payment channels: a customer pays a merchant repeatedly off-chain, and the merchant learns “no information beyond the fact that a valid payment ... has occurred on a channel that is open with them.” Then very little happened. BOLT never shipped on Zcash [14]; its successor zkChannels [3] never left DRAFT status; the implementation, libzkchannels [4], was archived in February 2023 as a proof of concept. There is no systematization of the object, no formal definition of it as a distinct primitive, and — as far as a systematic search across TLA+, Why3, Rocq, Isabelle, and Lean can establish — no machine-checked proof of payment unlinkability for any channel or credit construction in any prover. Every privacy proof in this literature is pen-and-paper.

*Repository: <https://github.com/dmarzzz/zk-payments-confetti>. The definitions in this paper were produced and stress-tested under an agent-assisted workflow whose review protocol and full gate record are documented in the repository README.

The object did not stay dead; it stayed unnamed. Keyed-verification credit tokens — ACT [32] and ARC [37], both 2025 IETF drafts — are anonymous balances spendable at exactly one verifier, with double-spend prevented against the issuer’s nullifier database: a channel’s state-update loop with the escrow replaced by “the issuer was already paid.” ZK API Usage Credits [9] is a deposit, a monotone spend index, and provider-countersigned balance updates — a commenter on the thread makes the channel mapping nearly verbatim. These systems answer to the same security questions BOLT answered to, but there is no shared definition to check them against.

The absence matters because this field has already demonstrated what happens without one. A2L’s privacy model [34] passed S&P 2021 peer review; a year later Glaeser et al. [20] exhibited two encryption schemes that satisfy its definitions and yield “a completely insecure system.” The definitions, not the proofs, are the risk surface.

What this paper does. Three things.

1. **Defines the object** (§2): a zk payment channel is a tuple (Setup, Open, Spend, Redeem, Close, Dispute) over payers, payees, and an idealized ledger, together with seven security games — no overspend, balance security for each side, spend unlinkability with an abort/evict adversary, closure liveness, priced divergence, and exculpability under collusion. The unlinkability game gives the adversarial payee BOLT §1.4’s abort and eviction powers, because a game without them proves a guarantee a real counterparty walks around. The definition is scheme-agnostic; this is the contribution, and the rest of the paper exists to defend it.
2. **Places the object** (§3) against the modern lineage — credit tokens, the hub-privacy line, ecash — with BOLT as origin, stating per system which property of the definition it lacks.
3. **Formalizes and machine-checks** (§4–5): two instantiations — a flat-ticket RLN credit protocol, arguably the smallest machine-checkable unlinkability target in the literature, and a refund-bearing variant covering variable-cost metering — with the safety, liveness, fleet, and *privacy* theorems for the flat-ticket instantiation kernel-checked in Lean 4 over VCV-io [35]: the spend-unlinkability advantage is proved exactly zero and the framing probability at most $1/|F|$, alongside a built-in calibration pair that separates the broken and fixed refund designs. §5 states exactly what is proved and what is not.

One methodological remark belongs up front. The definition in §2 is revision eleven. Eleven independent review rounds against earlier revisions produced concrete counterexamples against ten successive revisions — protocols and adversary schedules, not quibbles — and three of the resulting repairs (gateway-bound messages, close-time netting for the refund variant, verifiable spend counts at close) change protocol behaviour, not merely its description. We present these as design requirements discovered by adversarial definition review, with the counterexamples that forced them, because any construction of this shape that omits them is broken in the demonstrated way. The full review record, with every counterexample, is in the repository [12].

Honest scope. The formalization covers the protocol layer over an idealized ledger and idealized cryptography, in the random-oracle model. It does not verify circuits, and §6 lists the leaks the theorems do not cover — including two, the spend-count-at-close side channel and within-epoch linkability, that are intrinsic to the design.

2 The object

This section mirrors the specification of record (`Spec.md` revision 11 in the repository [12]); the Lean definitions are traceable to it sentence by sentence, and it — not the Lean — is what a reviewer reads.

2.1 Setting and notation

Three kinds of principal: **payers** (members), **payees** (gateways — N of them may share one logical payee role), and an **idealized ledger** $\mathcal{L}$: a totally ordered, atomically executed transaction log that includes honest transactions within delay Δ and runs the contract logic (escrow pool, membership tree, dedup set, windows, slashing, automatic settlement) exactly as specified. Public parameters: flat price C (or per-spend cap $C_{\max}$), deposit D , epoch length T_e , per-gateway per-epoch budget b , fleet size N , end-to-end reconciliation lag L , dispute window $\tau > \Delta$. Monetary quantities and indices are naturals; the signal algebra lives in a prime field F_p .

A member’s long-term secret is $k \in F_p$ with identity commitment $cm = H_{id}(k)$. The RLN signal at

spend index i on message m is [31, 9]

$$a = H_a(k, i), \quad x = H_x(m), \quad y = k + a \cdot x, \quad nf = H_{nf}(a),$$

with all hashes domain-separated and H_x mapping into $F_p \setminus \{0\}$ — at $x = 0$ the signal is $y = k$, the secret outright, a degenerate case the formalization isolated and the deployment must exclude. Two well-formed signals on the same (k, i) with $x \neq x'$ are two points on a line: anyone computes $a = (y - y')/(x - x')$ and $k = y - a \cdot x$. One signal, at $x \neq 0$, is consistent with every candidate secret via a unique coefficient and reveals nothing. This asymmetry — machine-checked as `rln_recover_k` and `rln_single_point_hiding` — is the entire double-spend mechanism: reuse of an index is punished by public recovery of k and loss of the deposit, and the punishment is claimable by anyone, with no watchtower. Tickets additionally carry an epoch pseudonym $nf_e = H_e(k, e)$, linkable within an epoch by design (it is what rate limiting counts) and unlinkable across epochs.

Messages are gateway-bound. A spend message is a pair $m = (G, \hat{m})$ — serving gateway identity plus payload — and Redeem at G rejects tickets naming any other gateway. This is a repair, not a transcription: see §2.4.

2.2 The algorithms

Setup $\rightarrow pp$: CRS, hash descriptions, constants, the gateway roster (sweep-authorized payee identities); for the refund variant, the payee’s signature keypair and a re-randomizable encryption key whose decryption key is output to nobody.

Open $(pp, D) \rightarrow (cm, st_P)$: the payer samples k , posts (cm, D) ; the ledger appends cm to the membership Merkle tree and adds D to a *commingled escrow pool*. The pool is forced: before any slash, a nullifier is by design unattributable to any cm , so no per-channel draw is possible. Open is a public ledger event; hiding *that* a party opened a channel is out of scope (§6).

Spend $(pp, st_P, m) \rightarrow (t, st'_P)$: off-ledger; the payer emits a ticket $t = (\pi, root, e, nf_e, (x, y, nf))$ where π proves, in zero knowledge: membership of cm under $root$; *solvency* $(i + 1) \cdot C \leq D$ at its current index i ; and well-formedness of the signal and epoch pseudonym for (k, i, m, e) . Emission consumes the index. The honest retry rule after an abort is to re-send the bit-identical ticket — the same point on the same line, unslashable; switching messages at a used index is self-slashing.

Redeem (pp, st_G, t) : the payee verifies π , the current root, the epoch, the gateway binding, and the rate budget for nf_e ; then nullifier logic against its spent set: fresh nf — accept; bit-identical tuple — reject-duplicate (the abort-retry path); same nf , different x — *evidence*, forwarded to **Dispute**. In the fleet, gateways exchange accepted tuples within lag L , and a gateway that merges a conflicting tuple emits evidence at merge time — required behaviour; without it a cross-gateway double spend that never produces a third signal is never slashed.

Close: either side, any time. The payer’s close and the payee’s settlement differ per instantiation and carry the two deepest repairs (§2.4, §4). The payee side in the flat instantiation is a unilateral *sweep*: registered gateways submit redeemed tuples; the ledger dedups by nullifier and pays C per fresh one from the pool.

Dispute (pp, ev) : anyone holding $ev = (nf, (x, y), (x', y'))$ with $x \neq x'$ submits it; the ledger recovers a and k , checks $nf = H_{nf}(a)$ and $cm = H_{id}(k)$ for an open channel, freezes the channel, evicts cm from the tree (the root rotates, so the member’s future spend proofs fail fleet-wide), and opens a claims window in which registered gateways recover outstanding value — sweeps first, then documented conflicting acceptances, pro-rata within class — against the member’s remaining deposit, *but only for acceptances committed to the ledger before the slash*. Each gateway checkpoints a binding Merkle commitment to its accepted set at its own cadence (at least once per epoch); claims open a pre-slash checkpoint with a membership witness. The remainder goes to the evidence submitter as bounty. The checkpoint-provenance rule exists because after a slash k is public and anything not anchored to a pre-slash commitment could be minted by the fleet itself.

The specification distinguishes two kinds of slash, and the distinction is load-bearing for the privacy analysis (§6). An *identity-slash* is the one just described: triggered by a **Dispute** evidence pair, the ledger recovers k from the line algebra, k becomes public, and the full MC4 claims window runs. A *fund-slash* — the void-and-slash of a false unused-claim at close, a settlement-detected sweep-bar violation, or a B failed-upgrade forfeit — carries no evidence pair, so k stays hidden: the channel’s remainder stays in the pool

with no submitter bounty (a bounty would need k to enumerate the member’s other nullifiers), and ordinary sweeps continue. Only the identity-slash publishes k ; the close-dispute and settlement slashes do not.

2.3 The security games

Seven statements, stated over the algorithms with explicit quantifiers, adversary classes, and anti-vacuity witnesses; proof order $T1 \rightarrow \text{exculpability lemma} \rightarrow T2/T3 \rightarrow T5 \rightarrow T6 \rightarrow T4 \rightarrow T7$. We summarize; the specification gives each in full.

T1 — No overspend. Against an arbitrary PPT payer coalition controlling all payers and the scheduler, facing one honest payee (or a perfectly synchronized fleet, $L = 0$): the accepted value attributed to any secret k never exceeds D . Attribution is by the knowledge-soundness extractor. With lag $L > 0$ the invariant is genuinely false during the window — deliberately; the exact price is T6.

T2 — Payee balance security. An honest payee following the sweep protocol (including monitoring Dispute windows, and keeping its checkpoints current at payer-closes) settles exactly $C \cdot |T|$ for its accepted set T , against payers who close early, close with false unused-claims, or deliberately trigger their own slash. The upper bound is unconditional (ledger dedup plus per-ticket price).

T3 — Payer balance security. Against a malicious payee coalition (all N gateways) plus all other payers: an honest payer that emitted j tickets recovers at least $D - jC$ (flat; $D - jC_{\max} + R$ with held receipts in the refund variant), and no adversary produces Dispute evidence that slashes an honest payer except with negligible probability. Emission is the authorization event: a payee that takes a ticket and refuses service is paid for it, and the theorem states that abort-griefing cost explicitly rather than hiding it.

T4 — Spend unlinkability (the headline game). The adversary plays the payee — all N gateways, all payee keys, arbitrarily many corrupt payers. The challenger runs two honest candidates with equal deposits, opened in batch. Pre-challenge, the adversary drives both candidates freely: spends on messages of its choice, retries, receipt issuance (refund variant), and the *abort/evict powers* of BOLT §1.4 [21] — it may refuse service to a candidate from any point on, and in the refund variant withholding receipts can drive a candidate insolvent. At challenge time the adversary outputs a message *vector* $\vec{m}^* = (m_1^*, \dots, m_q^*)$ of its choice ($q \geq 1$); the game checks the current epoch e^* is fresh for both candidates and both are challenge-capable for q (open, unclosed, solvent for q more spends); a candidate evicted into insolvency makes the challenge return $\perp$ — the game charges eviction to the anonymity set, not the scheme, which is precisely the calibrated content of the abort attack. Otherwise the hidden candidate P_b emits the whole q -spend session $t_1^*, \dots, t_q^*$ inside e^* (sharing the one session pseudonym nf_{e^*}), the adversary receives the batch, *and the game ends*: the guess is a pure function of retained memory plus the challenge response. Advantage is $|\Pr[b' = b] - \frac{1}{2}|$, with b sampled at game start so $\perp$ -paths contribute exactly $\frac{1}{2}$. The session form — an adversary-chosen vector rather than a single spend — is itself a repair the external review forced: the single-spend game certifies only unlinkability of a member’s *first* spend of an epoch, and a scheme leaking a persistent cross-epoch tag on second-and- later spends would pass it while being lifetime-linkable for any member that spends twice in an epoch (the deployed fleet’s normal usage). The session challenge exercises the second-spend wire format and catches that class; what is certified is the unlinkability of a member’s *whole epoch session* to its identity and its other epochs, within-session linkage via nf_{e^*} remaining by design.

Two further features of this game are results of review, not first drafts. First, the natural game with post-challenge oracles is *unsatisfiable*: three distinguishers win it against every scheme including sound ones — replay the challenge through the retry oracle, probe which candidate exhausts solvency one index early, read the spend count off a later close. All three exploit the bit-dependent continuation, so the game terminates at challenge delivery, and the residual leak (aggregate spend count at close) is stated honestly in §6 rather than defined away. Second, the game carries a *calibration requirement*: instantiated on the refund variant with a static encrypted running total (the original design), a concrete distinguisher must win — it issues receipts with equal totals to both candidates and bit-matches the presented ciphertext against its own issuance transcript [15] — and instantiated on the re-randomized repair, all PPT advantage must be negligible. A game that cannot separate the broken variant from the fixed one is the wrong game; this pair is the built-in test that ours is not, and the broken direction must be a constructive term in the formalization, not an unproven gap.

T5 — Closure liveness. An honest closer settles by $t + \Delta + \tau$ exactly, against a counterparty that never appears; silence, garbage, and concurrent disputes cannot extend the bound.

T6 — Priced divergence (fleet). Against one corrupted member facing N honest gateways with end-to-end reconciliation lag L : total accepted value is at most

$$\lfloor D/C \rfloor \cdot C + f(L), \quad f(L) = N \cdot b \cdot (\lceil L/T_e \rceil + 1) \cdot C,$$

and a member with two conflicting accepted signals is slashed fleet-wide within L of the second acceptance. The deployment condition $f(L) < D$ bounds the maximal burst by one deposit. The theorem deliberately does *not* claim attacker unprofitability or universal recovery: recovery through the claims window is capped by the remaining deposit and gated on pre-slash checkpoints, and an exhaust-then-burst schedule leaves a near-empty remainder. The operational levers are sweep cadence and checkpoint cadence. This is the theorem that makes an eventually-consistent spent set safe to run: the async window is priced by the deposit, not trusted away.

T7 — Exculpability under collusion (fleet). $N - 1$ gateways pooling all transcripts, plus corrupt members, with adaptive spend and close oracles against one honest member, produce evidence that slashes it with probability at most negligible. The algebraic core: the coalition holds at most one point per line, and one point at $x \neq 0$ determines nothing about k ; producing a second point is computing k . This is what makes an automatic, anyone-can-submit slash safe against the protocol’s actual threat model — the fleet’s own operators.

2.4 Three repairs, with the counterexamples that forced them

The specification records twenty modeling choices; three changed protocol behaviour and generalize beyond this protocol. Each was found by a reviewer who did not write the definition, as a concrete adversary schedule against the previous revision.

Replay across payees (gateway-bound messages). In the first revision, a spend message was just the request payload. The counterexample: replay the bit-identical ticket at all N gateways. Same x , same y , same nullifier — no conflicting pair ever forms, no evidence exists, the slash clock never starts, and each gateway’s private spent set happily accepts its copy. Excess extraction is roughly $(N - 1) \cdot D$ against a claimed bound of order $r \cdot L \cdot C$ (numerical witness from the review record: $N=3$, $b=100$, $T_e=1d$, $L=1$ min, $D=1000C$ gives $2000C$ of excess against a bound of $0.2C$). The repair binds the serving gateway’s identity into the hashed message, so cross-gateway reuse of an index *forces* distinct x — a conflicting pair — and the detection argument becomes sound. The general lesson: in any multi-verifier deployment of a nullifier scheme, replay across verifiers is value-bearing unless the verifier identity is inside the proof’s message. Three independent reviewers found this; it is the record’s canonical wrong-definition-nearly-proved case.

The gap-index close (verifiable spend counts). Through five revisions, a payer closed by declaring its spend count, with understatement policed by collision with its own spent indices. The fifth-round counterexample: nothing enforces index *contiguity* — indices are hidden witnesses — so a payer skips index 0, spends at indices $1..m$, and closes declaring index 0. The declaration collides with nothing, is undisputable, and recovers the full deposit after consuming the service. The root cause is structural: the ledger has no verifiable spend count. The repair is different per instantiation, and the difference is itself a finding (§4): the flat instantiation closes by *enumerating* PRF-fresh nullifiers of claimed-unused indices (false claims are disproven by bit-match against a pre-close checkpoint; genuinely unused nullifiers are PRF-hidden before the close reveals them, so no pre-close checkpoint can contain them — the same mechanism protects honest closers from fabricated disputes); the refund instantiation certifies the count inside the receipt chain and proves each spend’s index equals its receipt’s count, making spends contiguous by construction, then reveals the nullifier of the first index *beyond* the declared count so that closing on a stale receipt self-convicts.

Fund conservation under refunds (close-time netting). The flat instantiation lets gateways sweep per accepted nullifier. The second-round counterexample proved this cannot coexist with refunds: sweeps pay $C_{\max}$ per nullifier *and* the close pays the refund total $R - D + R$ out of a D deposit — and pre-slash unattributability makes per-channel netting impossible, so payee balance security, payer balance security, and pool solvency are jointly unsatisfiable. The repair removes unilateral sweeps from the refund variant

entirely: each channel settles exactly once, at close, with the payer receiving $(D + R) - j \cdot C_{\max}$ and the payee $j \cdot C_{\max} - R = \sum_{\ell} c_{\ell}$, under two circuit-enforced caps ($R \leq j \cdot C_{\max}$, else a colluding payee signs inflated refunds and drains the pool; $j \cdot C_{\max} \leq D + R$, else an overstated count pays the payee unboundedly). Conservation is then exact by construction. The general lesson: refund privacy and unilateral per-ticket settlement are incompatible under a commingled escrow; a refund-bearing anonymous channel must settle per channel, at close.

2.5 Model boundary

We formalize the protocol layer over the idealized ledger and idealized cryptography, in the random-oracle model. The NIZK relation is the mathematical statement above; the fidelity of any circuit to it is out of scope — the repository’s own wording is that anyone claiming it verifies SNARKs is misreading it. Named assumptions (knowledge soundness, zero knowledge, PRF/ROM with domain separation, EUF-CMA receipts, re-randomizable encryption, and a separately named `single_signal_hiding` — the additive, KDM-flavoured key use in $y = k + a \cdot x$ is not implied by standard PRF security) are confined to one audited registry file. Explicitly out of scope: circuit correctness, network-level timing and content fingerprints, funding-graph leakage at `Open`, a global passive adversary, relationship anonymity (the transport layer’s property), and the policy stake of the source construction.

3 Placement

Table 1 compares against the modern lineage, with BOLT as origin and — via its §1.4 abort attacks — the sharpest transferable threat model. “Ours” means the object of §2: recipient-bound anonymous spending with balance security both sides, detect-and-slash double-spend handling, and unlinkability against a payee holding abort/evict powers. Rows are sourced from a verified literature sweep (`RESEARCH.md` in the repository [12]); each cites its primary document.

Table 1: Placement against the lineage.

System	What it is	What it lacks relative to the §2 object
Chaum blind signatures [7]; Chaum–Fiat–Naor [8]	Bearer ecash; CFN adds offline double-spend detection revealing the cheater	Custodial mint; no escrow or dispute game; no balance-security theorems against the verifier; detection reveals identity but recovery is out of band
Compact E-Cash [5]	$O(1)$ wallets, serial-number nullifiers, identity extraction on reuse — BOLT’s direct ancestor	Same custodial shape; no channel state, no close/dispute, no priced asynchrony
BOLT [21]	Named the object: uni/bidirectional anonymous channels, revocation punishment, exculpability; §1.4 states the abort attacks	Punishment requires chain-watching (watchtowers); amounts visible to the merchant; anonymity conditional on anonymized funding; no machine-checked statement; abort attacks stated, not carried into the security games
zkChannels [3]	BOLT over an abstract arbiter, Pointcheval–Sanders signatures, an unlinking protocol for the open	Draft; dispute forfeits the entire balance to the merchant (a griefing cliff our close/dispute avoids); both sides online-or-watchtowered; archived implementation [4]
TumbleBit [24]	Unidirectional payment hub; the field’s reference anonymity accounting (k = completed payments per epoch)	Hub topology — a third party that is neither payer nor payee; fixed denominations; unlinkability vs the hub, not vs the payee
A2L [34] / A2L+ [20]	Adaptor-signature hub; A2L+ repairs the 2021 model after definitional counterexamples	Hub topology, hub adversary; the 2021 episode is this paper’s cautionary tale rather than its competitor
BlindHub [30]	Variable-amount hub unlinkability via blind adaptor signatures	Hub topology; solves amount-as-linking-tag, which flat pricing avoids by construction

System	What it is	What it lacks (cont.)
Accio [18]	Hub unlinkability, variable amounts, no NIZKs, no pre-locking	Hub topology; payer side plaintext; no payee-adversary unlinkability
Adaptor-signature foundations [19]	Formal definitions for the hub line’s main tool	A primitive, not a channel; listed because it is where that line’s definitional rigour now lives
Cashu [6] / Fedimint [17] / Taler [13]	Deployed Chaumian ecash: single mint / BFT federation / exchange with unlinkable change	Online prevention needs a synchronous spent set (Fedimint pays consensus latency on the spend path); custodial; no dispute game, no payee-balance theorem against the mint
Privacy Pass [10]	Standardized single-use unlinkable tokens	Unit tokens, not balances; replay defense still needs a per-origin spent list; no value semantics
ARC [37]	Keyed-verification N -use credential, presentations pairwise unlinkable	Issuer = verifier with online prevention: no escrow, no dispute, no slash; nothing prices a stale verifier view (our T6 has no analogue)
ACT [32]	ARC with money semantics: hidden balance, nullifier per spend, blind change — the closest living relative	Prevention against the issuer’s DB, so nothing to say about multiple verifiers, asynchrony, or framing; no exculpability game (the issuer’s DB is trusted); no close/liveness
Nym zk-nym [29]	Threshold-issued ticketbooks (Coconut lineage [33]), spendable at any gateway, deferred Bloom-filter reconciliation	Closest deployed shape to our fleet setting, but reconciliation cadence, synchrony assumptions, and double-spender penalties are undocumented — exactly the parameters our T6 makes explicit and proves against
ZK API Usage Credits [9]	Deposit + monotone index + RLN detect-and-slash + refund tickets; the construction of record for our refund variant	No formal definition or games; settlement cadence underspecified; the self-slash race and gap-index close (§2.4) live in the gap between its prose and a full specification
PrivateX402 [28]	One deposit funding N recipient channels; allocation privacy vs chain observers	No unlinkability vs the recipient at all (stable session key, cumulative receipts) — the property that defines our object
Nirvana [26] / RRTE [27]	Anonymous zero-confirmation payment guarantees; double-spender revealed by threshold decryption	Threshold committee in the loop; retail topology; no channel state or close

Two structural observations the table compresses. First, the hub line (TumbleBit through Accio) solves a different problem: it hides who-pays-whom from a *third party*; in our setting the payee itself is the adversary, so the right ancestor is BOLT, not the tumblers. Second, the prevention/punishment split is the live design axis: ACT/ARC and the ecash systems prevent double-spends against a synchronous verifier database, which is exactly what a fleet of mutually distrusting verifiers cannot cheaply have; BOLT punished via revocation, which needs watchtowers; the RLN line makes punishment cryptographic-economic and watchtower-free, at the price of a detection window that T6 prices.

On the verification side: Lightning has a UC treatment [25], machine-checked fund safety in Why3 [16], and TLA+ model-checking made feasible by proven refinements [22] — balance security is covered ground. Privacy is not: the mature game frameworks are SSProve [1] (Rocq) and CryptHOL [2] (Isabelle), and none of them has been used on channel unlinkability. We work in Lean 4 over VCV-io [35], which with ArkLib [36] is already verifying SNARK components — the ecosystem this protocol stack would eventually rest on.

4 The instantiations

A: flat-ticket RLN credits. Deposit D , flat price C , solvency $(i + 1) \cdot C \leq D$, per-index nullifier, slash on reuse. No refunds, no revocation, one inequality; **Spend** is a single message and the payee holds no

per-payer state, so a payee abort is exactly a denial of service. This is the protocol a Tor egress fleet [11] runs — near-uniform request cost makes the refund machinery deletable — and it is deliberately the smallest instantiation with all seven games defined on it. The fleet theorems T6/T7 exist only here: N gateways with local spent sets, gossip within L , merge-time evidence.

B: refund-bearing credits. The variant the source construction [9] needs for LLM-style cost variance (the thread’s argument: without refunds, a budget covering worst-case cost overshoots the mean by orders of magnitude). Per accepted spend the payee declares actual cost $c \leq C_{\max}$ and certifies a receipt chain over the triple (tag, R, n) — chain tag binding receipts to the channel (without it, receipts farmed on a cheap channel splice into another channel’s solvency proofs, a first-round counterexample), running refund total R , and certified spend count n . Solvency becomes $(i+1) \cdot C_{\max} \leq D + R$, proven against the certified ciphertext, with the spend’s index proven equal to n . Both representations of the certified total are formalized: **B-static** (present the last-signed ciphertext bit-identically — the original design, broken: the payee bit-matches it against its own issuance transcript [15], and the genesis receipt even links the first spend to the identity) and **B-rerand** (re-randomize and prove equivalence in zero knowledge — the patch). The unlinkability game *does* fail on B-static (a concrete distinguisher at advantage $\frac{1}{2}$) and pass on B-rerand (advantage 0); this calibration pair is machine-checked (§5), the built-in evidence that the game is not vacuous.

The close asymmetry, as a design-space observation. The gap-index counterexample (§2.4) hit both instantiations, and they cannot share a repair, because their information structures differ. A’s spending is non-interactive — one message, no countersignature — so no certified count can exist, and the close must *reveal*: enumerate the unused indices’ nullifiers, sized $\lfloor D/C \rfloor - j$, with disputes by checkpoint bit-match. B’s spending is interactive, so the count can be certified in the receipt chain and the close is $O(1)$: present the latest receipt, reveal one nullifier past the count. Non-interactive spending buys abort-immunity and no per-payer payee state, and pays for it at close time with an enumeration proportional to the unspent budget; interactive receipts buy certified counts and cheap closes, and pay with a live abort lever (withheld receipts stall solvency). We know of no prior statement of this trade-off, presumably because no prior design was pushed through an adversarial review that forced both closes to be sound.

C: the unidirectional nullifier-chain channel. A third instantiation, due to Vitalik Buterin (communicated via dmarzzz¹), sits at a different point in the design space and is markedly simpler; we record it here and formalize its core alongside A and B. Alice maintains a nullifier chain $N_{i+1} = H(N_i, c)$ under a private seed c . **Open** deposits D , names the recipient publicly, and commits (hiding) to N_1 . Each payment reveals the parent state’s committed next-nullifier and carries a ZK proof that the parent is either the on-chain genesis or *some* Bob-countersigned state — the signature verified inside the proof, so *which* state stays hidden — with $parent_balance + \delta = new_balance \leq D$, balances hidden, δ public; the message commits to the new balance and a fresh next nullifier, and Bob countersigns unless he has seen the revealed nullifier before. **Close** opens the closed state’s committed next-nullifier on chain; Bob challenges by exhibiting a message that revealed the same nullifier — a collision proves the closed state was extended — and a successful challenge forfeits the whole deposit to him. The one mechanism, the nullifier chain, does both jobs: duplicate detection during payments and stale-close detection at close, uniformly down to the genesis-refund case.

The comparison is instructive in both directions. The design is unidirectional, per-pair, and interactive (every payment needs a countersignature — B’s abort lever, not A’s abort-immunity), and its base form leaks D and the final split at the channel boundaries; recipient anonymity, deposit privacy, and close privacy are add-ons (an ephemeral per-channel key, or shielded-pool-integrated opens and closes), and it inherits the recipient-boundness of §6 unchanged. In exchange it deletes machinery this paper spent sections on: no epochs, no rate-limiting algebra, no fleet, no refund cascade — and, most notably for §6, *no identity-slash*. Its penalties are fund-forfeit only, so the retroactive deanonymization that FRAME must price in the RLN design (a published k unlocks the member’s entire request history) has no analogue here: a cheating Alice loses money, not her lifetime privacy, and the FRAME-shaped question collapses to the collision soundness of the challenge — an honest Alice’s latest-state close opens a nullifier no message ever revealed. The anonymity claim is also differently scoped: per-request unlinkability toward the recipient (Bob cannot link two payments to a sender or channel), where T4 is population unlinkability toward a payee across members

¹<https://gist.github.com/dmarzzz/ddcd1302c5f511001f8f46102874a08e>; archived in the repository as `research_knowledge/vitalik-nullifier-chain-channel.md`.

— hidden balances are what make it hold (a visible cumulative balance would let Bob rejoin the chain from his own δ records, which high-entropy δ makes easier, not harder). The construction is also natively post-quantum (hashes and STARK-friendly signatures; no curves), which none of A/B’s reference wires are. Its Lean formalization (`Zkpc/Chain/`) targets the same split this paper uses everywhere: safety and stale-close collision detection as Class-A invariants, per-request anonymity as a Class-B coupling, with the STARK and signature scheme idealized at the same boundary as A and B’s proofs.

5 Results: what is machine-checked

Everything in this section refers to Lean declarations in the repository [12], building with zero `sorry` under CI that also forbids `axiom` outside the assumptions registry and forbids `admit/native_decide`. The development in fact contains *no axiom declarations at all*: each named assumption is discharged by construction in the idealized model (knowledge soundness as transition guards — an accepted ticket *is* its extracted witness; zero knowledge as proof-free adversary views; hashes as lazily sampled random oracles), so `#print axioms` on every theorem shows only Lean’s `propext/Quot.sound/Classical.choice`. This was audited declaration by declaration (K2): the unlinkability, framing, calibration, and refund theorems each reduce to exactly those three standard Lean axioms and nothing more. The trust surface is the definitions, which is where the review effort went.

The model boundary bears repeating before the list: idealized ledger, random-oracle model, no circuits. These are theorems about the protocol layer.

Machine-checked theorems (flat-ticket instantiation).

- **T1** — `T1_no_overspend` (`Zkpc/Core/T1.lean`): accepted value per secret never exceeds D , single honest payee, arbitrary payer coalition.
- **Exculpability lemma, symbolic form** — `honest_never_slashed` (`Zkpc/Core/T1.lean`): a protocol-following payer is never slashed in any reachable state; the invariant carrier is `reach_inv`.
- **T2** — `Zkpc/Core/T2.lean`: unconditional upper bound `T2_upper` / `T2_paid_exact` (ledger dedup plus per-ticket price cap the payee’s revenue), collectability `T2_collectable` (from any reachable state a finite sweep sequence settles every accepted-unswept ticket whose window is open, with no payer cooperation), and `T2_settles_exactly`.
- **T3** — `T3_settled_amount`, `T3_payer_balance_security` (`Zkpc/Core/T3.lean`): the settled amount satisfies $\text{paid} + j \cdot C = D$ exactly (stronger than the specification’s floor), plus no-framing via the exculpability lemma.
- **T5** — `T5_payer_close_liveness`, with persistence `settleClose_stable` and progress `tick_progress` (`Zkpc/Core/T5.lean`): settlement is enabled from window expiry, stays enabled under every adversarial action, and a settling continuation exists from every reachable state; the payee half is T2’s collectability.
- **T6** — `T6_priced_divergence`, `T6_accept_count`, `T6_slash_within_L` (`Zkpc/Fleet/T6.lean`), on the counting lemmas `card_le_solvency_of_conflictFree` and `card_le_rate_window` and the epoch-straddle lemma `epochs_in_window` (`Zkpc/Fleet/Basic.lean`): accepted value $\leq \lfloor D/C \rfloor \cdot C + Nb(\lceil L/T_e \rceil + 1)C$ and fleet-wide slash within L . The formalization surfaced two boundary facts the prose missed: the ticket-count form is false at $C = 0$ (a zero-price fleet accepts unboundedly many non-conflicting tickets), and $T_e > 0$ is load-bearing.
- **RLN algebra** — `Zkpc/Games/RLN.lean`: `rln_recover_a`, `rln_recover_k` (two points reveal the secret — Dispute’s completeness), `rln_single_point_hiding` (one point at $x \neq 0$ is consistent with every candidate secret via a unique coefficient — the information-theoretic core of `single_signal_hiding`), `rln_evidence_complete`, `rln_evidence_sound` (the ledger’s slash predicate is satisfiable only by a genuine two-point exposure), and `rln_x_zero_degenerate` (the $x = 0$ counterexample that forced the domain-separation requirement).

Machine-checked privacy theorems (the headline). The two privacy games are defined over the advantage framework of `Zkpc/Games/Framework.lean` (`guessGap`, the $|\Pr[b' = b] - \frac{1}{2}|$ form; the challenge-terminated adversary shape `ChalAdversary`, whose post-challenge oracle silence holds by type; the abort/evict wrapper `withEvict`), and both are proved.

- **T4** — **spend unlinkability, exactly zero advantage** — `T4_flat_unlinkability` (`Zkpc/Games/T4.lean`), over the session-form game `unlinkGame/unlinkAdvantage` (`Zkpc/Games/Unlink.lean`): for the flat-

ticket ideal instantiation, *every* UNLINK adversary at *every* deposit budget has advantage = 0 — information-theoretically perfect unlinkability of a member’s whole epoch session, against a payee with the full pre-challenge abort/evict power of the BOLT §1.4 oracle. To our knowledge this is the first machine-checked spend-unlinkability result for any payment-channel or credit construction. The two honest residues (within-session nf_{e^*} linkage, MC6; the spend count at close, MC15) are pinned, not hidden: `flat_closeViewSimulatable` proves the close view is simulatable from $(cm, count)$ alone, so the leak is exactly the count and no more.

- **T7 — exculpability (framing) bound** — `T7_frame_bound` (`Zkpc/Games/T7.lean`), over `frameGame/frameWinProb` and the win predicate `Slashes` (deliberately stronger than `Dispute`): under the stated random-oracle good-event hypothesis `hobliv` (the adversary’s evidence is independent of the secret k), the probability that an $N - 1$ -gateway coalition frames an honest member is at most $1/|F|$ — the blind-guess floor. The query-term accounting that would discharge `hobliv` is now itself mostly kernel-checked infrastructure rather than prose: structural per-channel query budgets (`FrameQueryBounds`), the adaptive uniform-secret first-hit bound `uniformSecretProbeBound` ($q/|F|$), the adaptive multi-target slope-preimage bound `uniformSlopeProbeBound` ($q_{Nf} q_{sig}/|F|$), a quantitative real-to-ideal assembly theorem `T7_frame_bound_of_pointwise`, and the composition endpoint `T7_frame_query_bound`, which turns a deferred-sampling certificate into the corrected concrete bound $(q_A + q_E + q_{Id} + q_{Nf} q_{sig} + q_{sig}^2 + 1)/|F|$. An exact audited ghost of the real handler (`FrameAudit`: bad-event monotonicity, per-step resource growth, projection to `frameImpl` on every adaptive run) and a secret-independent ideal handler with a canonical secret-erasing state map and per-oracle idealization step lemmas (`FrameIdeal`) are proved. The certificate statement itself was then stress-tested and *refuted* as originally frozen (`frameDeferredSampling_refuted`, `Zkpc/Games/FrameDeferred.lean`): a kernel-checked two-probe adversary — two H_{id} queries against the public commitment — forces any single secret-independent generator to carry mass $\geq 1 - 2/|F|$ on each of two disjoint slash events, so the pointwise-in- k domination demanded by `FrameDeferredSampling` is unsatisfiable for $|F| > 5$. This is a gate finding in the repository’s sense (the definition, not the theorem, was wrong): the leak mass is bounded only *averaged over the uniform secret*, which is also exactly the quantity the FRAME experiment consumes. The corrected socket `FrameDeferredSamplingAvg` composes to the *same* final bound via `T7_frame_query_bound_avg`, and the pointwise form strictly implies it. Honestly in-place: constructing the averaged certificate from `frameImpl` by the stateful transcript coupling remains open, and we do not claim the unconditional bound — but the pointwise route is now provably a dead end rather than an open problem, which is itself information. Three *must-win* calibration adversaries confirm the game has teeth: `frameWinProb_YK_eq_one` (degenerate $y = k$), `frameWinProb_aReuse_eq_one` (a reused across indices), and `frameWinProb_slopeReveal_eq_one` (the formal witness that the slope-preimage channel is real, so the $q_{Nf} q_{sig}$ term is required, not conservative slack) each frame with probability exactly 1.
- **The calibration pair (the built-in definitional test)** — `unlinkAdvantage_staticDistinguisher_eq_half` (the broken B-static design: a concrete distinguisher wins at exactly $\frac{1}{2}$) and `unlinkAdvantage_bRerand_eq_zero` (the B-rerand repair: every adversary at exactly 0), in `Zkpc/Games/Calibration.lean`. The *same* game separates the broken variant from the fixed one, which is the property a wrong game lacks. The must-catch battery joins it: `unlinkAdvantage_aIndexLeak` (index-in-clear, A’s first calibration point), `unlinkAdvantage_nfeReuse` (nf_e derived without e), and `unlinkAdvantage_multTagDistinguisher_eq_half` (the multiplicity-tag scheme the session form was introduced to catch, won by a $q = 2$ distinguisher) — each at exactly $\frac{1}{2}$. The algebraic core T7 reduces to `(rln_single_point_hiding, rln_evidence_sound, Zkpc/Games/RLN.lean)` is in the flat-ticket list above.

Machine-checked refund-variant safety, cascade, and fleet aggregation. The refund variant (instantiation B) is machine-checked in `Zkpc/Refund/`: `T1_B_no_overspend` (accepted cost $\sum_{\ell} c_{\ell} \leq D$), `T3_B_floor` (a cooperatively-settled payer recovers exactly $D - \sum_{\ell} c_{\ell}$ and is provably not slashed), `conservation` (every settled channel splits exactly D between the two parties, cooperative close and fund-slash forfeit alike), and `self_slash_race_closed` (settlement happens at most once and no path strands funds, so the self-slash race cannot leave the payee short). The two deferrals the previous revision of this paper carried here are now discharged. `Zkpc/Refund/Cascade.lean` models the full failed-upgrade cascade — successive withheld-receipt upgrades, one count restored per round: upgrade claims never overshoot the certified count (`cascade_upgrades_le_understatement`), a terminal cascade has settled

(`cascade_terminal_settled`), settlement happens at exactly the true count with exactly $n - j$ upgrades (`cascade_settled_upgrades_eq`), and the final payouts conserve D (`cascade_final_payouts`), with an executable driver (`execCascade_progress`). `Zkpc/Refund/Fleet.lean` lifts the single-channel results to interleaved multi-channel reachability and aggregates no-overspend, settlement conservation, and the cooperative payer floor across any finite fleet (`fleet_no_overspend`, `fleet_conservation`, `fleet_payer_floor`). `Zkpc/Fleet/Recovery.lean` formalizes the fleet-side post-slash recovery rule (MC19): pre-slash checkpoint eligibility (`preSlash_claim_eligible`, `postSlash_claim_ineligible`), sweep-before-conflict seniority, remainder-capped payouts (`identityRecovery_capped`), exact conservation (`identityRecovery_conservation`), full recovery when eligible demand fits (`identityRecovery_all_full`), and the distinct fund-slash forfeit path (`fundSlashRecovery_full`).

Machine-checked wire-protocol bridges (the O1–O4 obligations, discharged). The gap the previous revision stated as an obligation — T4 is proved on a proof-free view, the real wire carries a NIZK — is now closed for three concrete proof-bearing wire encodings, each with a kernel-checked *zero-loss* instance of `zkBridgeObligation` (`Zkpc/Games/FullTicketInstance.lean`, `Zkpc/Games/SigmaInstance.lean`): a *masked-proof wire* (the honest prover retains a private witness and emits it under a fresh additive one-time mask; `evalDist_spendBatch_maskedProof` proves the witness-dependent real transcript equals the simulator distribution exactly, giving `T4_maskedProof_unlinkability` and `maskedProof_zkBridge`); an *interactive Sigma wire* (`Zkpc/Crypto/LinearSigma.lean`: verifier completeness, a simulator with exact transcript equality `evalDist_real_eq_simulated`, two-transcript special-soundness extraction `special_soundness`; `T4_sigmaFlat_unlinkability`, `sigmaFlat_zkBridge`); and a *lazy-ROM Fiat–Shamir wire* (`Zkpc/Crypto/FSRom.lean`: simulator distributions `evalDist_fsProveLazy_eq_simulated` plus explicit programming- and fork-collision bounds `fsProgramCollisionBound`, `fsForkChallengeCollisionBound`; `T4_fsFlat_unlinkability`, `fsFlat_zkBridge`). The refund-side B-instance obligations are likewise discharged (`Zkpc/Games/BInstances.lean`): the rerandomized challenge path (`bRerand_spendBatch_none_zero`, O2), adversary-issued genesis receipts and issuer receipt updates with capability monotonicity (`bIdeal_openCh_adversary_genesis`, `bIdeal_serve_issuer_receipt`, `bIdeal_serve_capable_mono`, M2/O3), and close-view simulatability for both B-static and B-rerand (`bIdeal_closeViewSimulatable`, O4). On the refund cryptography itself, `Zkpc/Crypto/MaskedEncryption.lean` proves exact distributional rerandomization and refund-update privacy for the additive masked cipher, and `Zkpc/Crypto/ReceiptMac.lean` proves a $1/|F|$ fresh-message forgery bound for the one-time algebraic receipt MAC (`mac_forgery_bound`). What remains beyond these information-theoretic reference layers is deployment-grade: a concrete hash-implementation reduction for the FS layer and multi-query EUF-CMA signature/MAC reductions (§6).

Executable refinement. The relational transition systems the theorems quantify over are now refined by executable operations: `Zkpc/Core/Refinement.lean` proves that executable open, honest spend, fresh redeem, payer close, identity dispute, arbitrary sweep lists, and the MC20 contract drivers (close dispute, successful settlement, settlement-time voiding) return traces of their corresponding `Step` constructors (`sweep_refines_trace`, `refined_steps_reachable`); `Zkpc/Refund/Refinement.lean` and `Zkpc/Fleet/Refinement.lean` do the same for refund accept/close/force-close and fleet tick/admission/slash. States generated by running the executable layer therefore inherit T1–T6 and the refund/fleet invariants by construction.

The multi-recipient network layer (the named open problem, definitional half). `Zkpc/Network/State.lean` defines a portable-deposit network — one deposit funding arbitrarily many recipients over a shared global nullifier set with recipient-directed settlement — and proves global deduplication (`global_dedup`), network-wide no-overspend (`no_overspend`), exact recipient-partitioned payout accounting, unrelated-recipient view isolation, and executable refinement. `Zkpc/Network/Credential.lean` gives the first concrete credential adapter — recipient, global nullifier, value, and payload bound into a Fiat–Shamir statement — with honest issuance verifying, verified fresh redemption refining to network admission, cross-recipient nullifier replay rejected (`redeem_rejects_global_replay`), and an end-to-end payment theorem composing verification, executable redemption, settlement, reachability, and shared-deposit no-overspend (`credential_payment_end_to_end`). `Zkpc/Network/Issuance.lean` adds a finite threshold-issuance reference: share aggregation correctness, perfectly hiding blind requests (`evalDist_blindRequest_uniform`, `issuerView_message_independent`), fork extraction (`ticket_fork_extract`), and exact recipient-view

simulation/unlinkability (`recipientView_unlinkable`, `recipientView_simulatable`).

One-trace composition. `Zkpc/Core/Composition.lean` bundles the guarantees that were previously separate endpoints. `channel_endToEnd_composition`: on a single reachable channel trace, an honest payer with a posted close reaches a settled successor state of the *same* trace at which, simultaneously, the close settles with the exact payer floor (T5+T3), every member satisfies no-overspend (T1), the payee is settled exactly (T2), and the honest closer is unslashed (exculpability). `wire_endToEnd_composition`: for the verified Fiat–Shamir wire family, perfect T4 unlinkability and the zero-loss ZK bridge hold together. Both are axiom-clean.

An earlier review round on the game files produced a fix list — the adversary’s view omitting the proof object (which would trivialize the zero-knowledge step), the refund genesis receipt needing the adversary in the loop, FRAME needing the close-time nullifier reveals — and every item on it was discharged before the proofs landed (`zkBridgeObligation` as the stated bridge, adversary-issued genesis, the `nfAt` superset, the `roId` commitment surface).

What stands today: every one of the seven security statements is machine-checked on the flat-ticket instantiation; the refund variant’s safety, conservation, failed-upgrade cascade, and finite-fleet aggregation are machine-checked; the O1–O4 model-to-real bridges are discharged with zero loss for three concrete wire encodings and the B instantiation; the executable layer refines the relational one; the calibration battery confirms the games are not vacuous; the channel and wire guarantees compose on one trace; and the whole tree is axiom-clean (K2). The one substantive in-model deferral left is stated where it occurs: T7’s unconditional bound awaits the secret-averaged handler coupling (`FrameDeferredSamplingAvg`), with the quantitative composition endpoint already kernel-checked and the pointwise certificate variant kernel-refuted. Everything else that remains is deployment-grade cryptography — concrete hash/signature reductions behind the ideal reference layers — and circuits are out of the model boundary (§2.5).

6 Honest limits

Recipient-boundness. The object binds a deposit to one logical payee. That is the defining restriction, shared with ACT/ARC and every keyed-verification design; the fleet setting relaxes it only by making N gateways one logical payee with an internal consistency problem (which T6 prices). It is not a multi-merchant payment system.

Capital lockup. D per payer–payee pair, locked for the channel’s life. Below a payment-frequency threshold a channel does not pay for its lockup [23]; per-pair channels against N verifiers multiply this cost by N , which is the argument for the fleet-as-one-payee shape.

Funding-graph leakage. Open is a public ledger event naming cm and D ; the deposit transaction links a funding address — and via exchange KYC trails, plausibly an identity — to membership. The sources prescribe shielded or Privacy-Pools-style funding; we do not model it, and without it the anonymity set is fiction at the funding edge. BOLT’s anonymized-capital requirement is the same point one decade earlier.

The spend-count-at-close side channel. Both closes reveal the spend count: as the certified j in the refund variant, as $\lfloor D/C \rfloor - |U|$ in the flat enumeration. T4 does not cover it — the game terminates at challenge delivery precisely because a close after the challenge leaks the count. A member that closes immediately after a distinctive burst correlates its count with observed traffic. Mitigations (delay closes; close on round counts) are operational, unproven here.

Within-epoch linkability, by design. All of a member’s spends in one epoch share nf_e : rate limiting *is* a linking mechanism at epoch granularity. T4 claims unlinkability across epochs and to identity, never within an epoch; the game’s freshness condition is that scope made formal. Epoch length is the privacy/throughput dial, and cross-epoch intersection attacks over stable memberships remain — as everywhere in this literature — an unpriced erosion.

A slash is retroactive deanonymization — identity-slash only. An identity-slash publishes k , and from k every $nf_i = H_{nf}(H_a(k, i))$ and every epoch pseudonym $nf_e = H_e(k, e)$ is enumerable — the protocol relies on this for post-slash sweep attribution. So an identity-slash does not cost the member D ; it retroactively links the member’s *entire lifetime request history* to its now-public cm . The privacy T4 delivers is

therefore contingent and revocable: it holds exactly as long as k stays secret, and the protocol itself contains the mechanism that publishes k . There is no forward anonymity by design — a legitimate trade (it is what makes the slash claimable by anyone, watchtower-free), but one a bare “unlinkability” claim would over-read. The blast radius is widest for accidental double-emission: an honest-but-buggy client that re-emits a different message at a used index self-slashes and loses both D and its history; T7 says nothing here, because the client did double-sign. Exculpability protects *correct* users, not merely honest ones, and this is the weight FRAME actually carries: it is the sole barrier between every member and total retroactive deanonymization by the fleet’s own operators. The distinction of §2 bounds the damage — a fund-slash (false-claim void, settlement bar, B failed-upgrade forfeit) keeps k hidden and links nothing; only the evidence-pair identity-slash deanonymizes. (Instantiation C is the design-space counterpoint: its penalties are fund-forfeit only, so this entire limit is absent there at the cost of interactivity and per-pair channels — see §4.) Correspondingly, FRAME machine-checks the identity-slash door; A’s close-dispute exculpability is covered by the Core exculpability lemma (`honest_never_slashed`), while B’s failed-upgrade exculpability remains a specification-level argument, not machine-checked.

The stale-close residue in B. Under receipt withholding, an honest B payer can be wedged one count behind its true spend count, and its only available close is on a stale receipt — which the revealed nf_j links to cm for that one epoch session (rev-11 F10-1, MC15). The upgrade sub-window restores the *funds* but not the *privacy*: the first close is on-ledger and permanent. The nuance is that the linkage is not the adversary’s to take freely — a payee declining to dispute keeps the withheld receipt unpublished and gains the linkage at the cost of one forgone $c \leq C_{\max}$, so publication prices the payee’s fund *recovery*, not the linkage; and the session extension of the linkage requires the ticket transcript, which in B only the payee holds. It is one session, not a lifetime, but it is a real residue and we state it.

From two candidates to the population, and the cost of aborts. T4 is a two-candidate game; deployments care about anonymity within the live membership. The standard hybrid does go through here — other members simulate as corrupt payers run honestly, since payers hold independent secrets, never interact peer-to-peer, and D is a global constant — so the $2 \rightarrow n$ reduction loses the usual factor and no more; we state the corollary but do not machine-check it, and note that with the adversary-supplied genesis in B the hybrid must be checked to respect the genesis stage (an unquantified erosion). The delivered guarantee is indistinguishability *within the challenge-capable set*, and that set is adversary-controlled at zero protocol cost: refusal of service in A, receipt withholding in B, each evicts a member from the set. After q evictions the set is $n - q$, down to 1, and nothing in T4 resists this — the defence is operational (members notice starvation and leave; gateways compete), not cryptographic. No theorem in T1–T7 constrains a payee’s right to refuse service; the protocol prices refusal at zero.

Window recovery presumes fleet honesty. Post-slash recovery pays registered gateways against pre-slash checkpoints. A member colluding with a corrupt gateway can pre-checkpoint real, cooperatively produced “service” at every solvent index and exhaust the slashed remainder senior to honest gateways’ claims. No theorem’s adversary class covers corrupt gateways in the *recovery* role; T7 protects members from gateways, not gateways from each other.

Close racing. Service accepted between a close’s ledger inclusion and its settlement cannot be attributed to the closing channel: in the refund variant the exposure is bounded by acceptance rate times τ (pause acceptance while a close window is open); in the flat variant an acceptance in flight at the close transaction is structurally un-checkpointable and the tardy gateway bears exactly its un-checkpointed tickets. Checkpoint cadence is each gateway’s own recovery lever, and T2 is conditioned on it being exercised.

T6 is a bound, not a business case. Recovery of the diverged value is remainder-capped and checkpoint-gated; an exhaust-then-burst member leaves little to recover. $f(L) < D$ bounds the burst by one deposit and no more.

$x \neq 0$. The signal at digest zero is the secret. The deployment must domain-separate H_x away from zero; the formalization carries this as an explicit hypothesis, machine-checked in both directions (`rln_single_point_hiding` requires it; `rln_x_zero_degenerate` shows why).

The named open problem: multi-recipient generalization. Everything in the channel object binds value to one logical payee. A member paying N *independent* payees with per-pair channels needs N deposits, N partitioned anonymity sets, and hands each payee BOLT’s abort lever — the three costs that made the per-pair shape lose to the fleet shape in our application analysis. The generalization needs either portable deposits (value that moves between payees without a linking event — the hub line’s problem, with the hub’s anonymity accounting) or threshold issuance over a shared spent structure (the Nym-shaped hybrid [29], whose reconciliation guarantees are exactly what would need the T6 treatment). PrivateX402 [28] shows what giving up costs: one deposit across N recipients, and every recipient links every request. The definitional and accounting half of this problem is now formalized (§5): the portable-deposit network machine, its credential adapter, and the threshold-issuance reference construction prove global deduplication, no-overspend, payout partitioning, recipient-view isolation, blind-request hiding, and recipient-view unlinkability. What we still do not claim is the composition that would make it a *solved* problem: an adaptive multi-session network game connecting those per-session distributions to the executable admission and settlement trace, and a production threshold-signature unforgeability reduction. The problem is now half-closed, and we say which half.

Also outside every theorem: static corruption only; no clock skew (a skew-tolerant model enlarges the T6 budget by a small constant); and the entire traffic-analysis surface — timing, size, content fingerprints — which in the source thread’s own discussion is “a richer fingerprint than the wallet address itself.”

7 Reproducibility

Repository: <https://github.com/dmarzzz/zk-payments-confetti>. Toolchain, pinned: `leanprover/lean4:v4.30.0`; `mathlib v4.30.0` (manifest revision `c5ea0035...`); VCV-io at commit `8f5dc4f2...` [35]. Build:

```
lake exe cache get && lake build
```

CI runs the build on the pinned toolchain and additionally fails on: `sorry` anywhere; `axiom` outside `Zkpc/Assumptions.lean`; `admit` or `native_decide` anywhere.

Table 2: Theorem-to-file map.

Statement	File	Declarations
T1	<code>Zkpc/Core/T1.lean</code>	<code>T1_no_overspend</code> , <code>reach_inv</code>
Exculpability (symbolic)	<code>Zkpc/Core/T1.lean</code>	<code>honest_never_slashed</code>
T2	<code>Zkpc/Core/T2.lean</code>	<code>T2_upper</code> , <code>T2_paid_exact</code> , <code>T2_swept_accepted</code> , <code>sweepOne_enabled</code> , <code>T2_collectable</code> , <code>T2_settles_exactly</code>
T3	<code>Zkpc/Core/T3.lean</code>	<code>payer_pay_inv</code> , <code>settleClose_enabled</code> , <code>T3_settled_amount</code> , <code>T3_payer_balance_security</code>
T5	<code>Zkpc/Core/T5.lean</code>	<code>T5_payer_close_liveness</code> , <code>settleClose_stable</code> , <code>tick_progress</code>
T6	<code>Zkpc/Fleet/T6.lean</code> , <code>Zkpc/Fleet/Basic.lean</code>	<code>T6_priced_divergence</code> , <code>T6_accept_count</code> , <code>T6_slash_within_L</code> , <code>card_le_solveny_of_conflictFree</code> , <code>card_le_rate_window</code> , <code>epochs_in_window</code> , <code>fleet_inv</code>
RLN algebra	<code>Zkpc/Games/RLN.lean</code>	<code>rln_recover_a</code> , <code>rln_recover_k</code> , <code>rln_single_point_hiding</code> , <code>rln_x_zero_degenerate</code> , <code>rln_evidence_complete</code> , <code>rln_evidence_sound</code>
Game framework	<code>Zkpc/Games/Framework.lean</code>	<code>guessGap</code> , <code>guessGap_eq</code> , <code>hiddenBitAdvantage_eq_half_boolDistAdvantage</code> , <code>hiddenBitAdvantage_const</code> , <code>hiddenBitAdvantage_eq_zero_of_distEquiv</code> , <code>ChalAdversary</code> , <code>withEvict</code>
T4 (unlinkability)	<code>Zkpc/Games/T4.lean</code> , <code>Zkpc/Games/Unlink.lean</code>	<code>T4_flat_unlinkability (= 0)</code> , <code>unlinkGame</code> , <code>unlinkAdvantage</code> , <code>UnlinkScheme</code> , <code>flat_closeViewSimulatable</code>
T7 (framing)	<code>Zkpc/Games/T7.lean</code> , <code>Zkpc/Games/Frame.lean</code>	<code>T7_frame_bound ($\leq 1/ F$ under <code>hobliv</code>)</code> , <code>frameWinProb_YK_eq_one</code> , <code>frameWinProb_aReuse_eq_one</code> , <code>frameWinProb_slopeReveal_eq_one</code> , <code>frameGame</code> , <code>frameWinProb</code> , <code>Slashes</code>

Statement	File	Declarations (cont.)
T7 query-budget composition	Zkpc/Games/T7.lean, Zkpc/Games/Frame{Audit,Ideal,DeferredSampling,DeferredSamplingAvg,T7_frame_bound_of_pointwise,T7_frame_query_bound,frameDeferredSampling_refuted,FrameDeferredSamplingAvg,T7_frame_query_bound_avg,auditedFrameImpl,idealFrameImpl,FrameCoupled,idealizeFrame	FrameQueryBounds, uniformSecretProbeBound, DeferredSamplingProbeBound, T7_frame_bound_of_pointwise, T7_frame_query_bound, frameDeferredSampling_refuted, FrameDeferredSamplingAvg, T7_frame_query_bound_avg, auditedFrameImpl, idealFrameImpl, FrameCoupled, idealizeFrame
Wire ZK bridges (O1)	Zkpc/Games/FullTicketInstance.lean, Zkpc/Games/SigmaInstance.lean	maskedProof_unlinkability, maskedProof_zkBridge, T4_sigmaFlat_unlinkability, sigmaFlat_zkBridge, T4_fsFlat_unlinkability, fsFlat_zkBridge, fullFlat_zkBridge
Sigma / Fiat-Shamir cores	Zkpc/Crypto/LinearSigma.lean, Zkpc/Crypto/FSRom.lean	completeness, evalDist_real_eq_simulated, special_soundness, evalDist_fsProveLazy_eq_simulated, fsProgramCollisionBound, fsForkChallengeCollisionBound
B-instance obligations (O2–O4)	Zkpc/Games/BInstances.lean	bRerand_spendBatch_none_zero, bIdeal_openCh_adversary_genesis, bIdeal_serve_issuer_receipt, bIdeal_serve_capable_mono, bIdeal_closeViewSimulatable
Refund crypto references	Zkpc/Crypto/MaskedEncryption.lean, Zkpc/Crypto/ReceiptMac.lean	evalDist_rerandomize_cipher_uniform, evalDist_refundUpdate_cipher_uniform, mac_forgery_bound
Refund cascade	Zkpc/Refund/Cascade.lean	cascade_upgrades_le_understatement, cascade_settled_upgrades_eq, cascade_terminal_settled, cascade_final_payouts, execCascade_progress
Refund fleet aggregation	Zkpc/Refund/Fleet.lean	fleet_no_overspend, fleet_conservation, fleet_payer_floor
Fleet recovery (MC19)	Zkpc/Fleet/Recovery.lean	identityRecovery_conservation, identityRecovery_capped, identityRecovery_all_full, preSlash_claim_eligible, postSlash_claim_ineligible, fundSlashRecovery_full
Executable refinement	Zkpc/{Core,Refund,Fleet}/Refinement.lean	insert_refines_trace, refined_steps_reachable, exec*_refines_step, exec_step_reachable
Multi-recipient network	Zkpc/Network/{State,Credentials,PaymentEnd,GlobalDedup,RedeemRejectsGlobalReplay,CredentialPaymentEndToEnd,BlindRequestUniform,TicketForkExtracts,RecipientViewUnlinkable}	channel_endToEnd_composition, wire_endToEnd_composition
One-trace composition	Zkpc/Core/Composition.lean	channel_endToEnd_composition, wire_endToEnd_composition
Calibration	Zkpc/Games/Calibration.lean	unlinkAdvantage_staticDistinguisher_eq_half, unlinkAdvantage_bRerand_eq_zero, unlinkAdvantage_aIndexLeak, unlinkAdvantage_nfeReuse, unlinkAdvantage_multTagDistinguisher_eq_half
Refund variant (B, N=1)	Zkpc/Refund/Safety.lean, Zkpc/Refund/State.lean	T1_B_no_overspend, T3_B_floor, conservation, self_slash_race_closed
Assumption registry	Zkpc/Assumptions.lean	Named, dischargedBy (no axiom declarations exist)
State machines	Zkpc/Core/State.lean, Zkpc/Core/Flat.lean, Zkpc/Fleet/Basic.lean	transition systems the above quantify over

The specification of record is `Spec.md` (revision 11); every Lean definition is traceable to it, and the full adversarial review record — eleven rounds against the specification, three against the Lean games, plus independent statement (K1), axiom (K2), and external-cryptographer (K4) audits, with every counterexample — is `research_knowledge/gates.md`. TLA+ models of the flat and fleet state machines, including ablation configurations that replay the gateway-binding and merge-evidence counterexamples (`tla/ZkpcFleetNoBind.cfg`, `tla/ZkpcFleetNoMergeEv.cfg`), are in `tla/`.

References

- [1] Carmine Abate, Philipp G. Haselwarter, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Catalin Hritcu, Kenji Maillard, and Bas Spitters. SSProve: A foundational framework for modular cryptographic proofs in Coq. ePrint 2021/397; ACM TOPLAS, 2021.
- [2] David A. Basin, Andreas Lochbihler, and S. Reza Sefidgar. CryptHOL: Game-based proofs in higher-order logic. ePrint 2017/753, 2017.

- [3] Bolt Labs. zkChannels protocol specification, v0.3.2 (DRAFT), 2021.
- [4] Bolt Labs. libzkchannels (archived February 2023), 2023.
- [5] Jan Camenisch, Susan Hohenberger, and Anna Lysyanskaya. Compact E-Cash. ePrint 2005/060; EUROCRYPT 2005, 2005.
- [6] Cashu project. NUT-00: Notation, utilization, and terminology, 2022.
- [7] David Chaum. Blind signatures for untraceable payments. In *Advances in Cryptology: Proceedings of CRYPTO '82*, 1983.
- [8] David Chaum, Amos Fiat, and Moni Naor. Untraceable electronic cash. In *Advances in Cryptology: CRYPTO '88*, 1990.
- [9] Davide Crapis and Vitalik Buterin. ZK API usage credits: LLMs and beyond. ethresear.ch, 2026.
- [10] A. Davidson, J. Iyengar, and C. A. Wood. The Privacy Pass architecture. RFC 9576, 2024.
- [11] dmarz. Reputation-gated onion-service egress, 2026.
- [12] dmarz. zk-payments-confetti: Specification, gate record, and Lean formalization, 2026.
- [13] Florian Dold, Sree Harsha Totakura, Benedikt Müller, Jeff Burdges, and Christian Grothoff. Taler: Taxable anonymous libre electronic reserves. In *Proceedings of SPACE*, 2016.
- [14] Electric Coin Company. BOLT: Private payment channels, 2016.
- [15] Omar Espejel. Linkability simulation for static encrypted refund totals. gist, 2026.
- [16] Grzegorz Fabiański, Rafał Stefański, and Orfeas Stefanos Thyfronitis Litos. A formally verified Lightning Network. arXiv:2503.07200, 2025.
- [17] Fedimint contributors. Fedimint: Federated e-cash, 2022.
- [18] Zhonghui Ge, Jiayuan Gu, Chenke Wang, Yu Long, Xian Xu, and Dawu Gu. Accio: Variable-amount, optimized-unlinkable and NIZK-free off-chain payments via hubs. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023. ePrint 2023/1326.
- [19] Paul Gerhart, Dominique Schröder, Pratik Soni, and Sri AravindaKrishnan Thyagarajan. Foundations of adaptor signatures. ePrint 2024/1809, 2024.
- [20] Noemi Glaeser, Matteo Maffei, Giulio Malavolta, Pedro Moreno-Sánchez, Erkan Tairi, and Sri AravindaKrishnan Thyagarajan. Foundations of coin mixing services. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.
- [21] Matthew Green and Ian Miers. Bolt: Anonymous payment channels for decentralized currencies. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017. ePrint 2016/701.
- [22] Matthias Grundmann and Hannes Hartenstein. Model checking the security of the Lightning Network. arXiv:2505.15568, 2025.
- [23] Paolo Guasoni, Gur Huberman, and Clara Shikhelman. Lightning Network economics: Channels. *Management Science*, 70(6), 2024.
- [24] Ethan Heilman, Leen AlShenibr, Foteini Baldimtsi, Alessandra Scafuro, and Sharon Goldberg. TumbleBit: An untrusted Bitcoin-compatible anonymous payment hub. In *Network and Distributed System Security Symposium (NDSS)*, 2017.
- [25] Aggelos Kiayias and Orfeas Stefanos Thyfronitis Litos. A composable security treatment of the Lightning Network. In *IEEE Computer Security Foundations Symposium (CSF)*, 2020. ePrint 2019/778.

- [26] Akash Madhusudan, Mahdi Sedaghat, Philipp Jovanovic, and Bart Preneel. Nirvana: Instant and anonymous payment-guarantees. ePrint 2022/872, 2022.
- [27] Akash Madhusudan, Mahdi Sedaghat, Samarth Tiwari, Kelong Cong, and Bart Preneel. Reusable, instant and private payment guarantees for cryptocurrencies. In *Australasian Conference on Information Security and Privacy (ACISP)*, 2023. ePrint 2023/583.
- [28] Conor McMenamin and Grigore Grigor. PrivateX402: Privacy-preserving payment channels for multi-agent AI systems. ethresear.ch, 2026.
- [29] Nym Technologies. zk-nym overview. Nym network documentation, 2023.
- [30] Xianrui Qin, Shimin Pan, Arash Mirzaei, Zhimei Sui, Oğuzhan Ersoy, Amin Sakzad, Muhammed F. Esgin, Joseph K. Liu, Jiangshan Yu, and Tsz Hon Yuen. BlindHub: Bitcoin-compatible privacy-preserving payment channel hubs supporting variable amounts. In *IEEE Symposium on Security and Privacy (S&P)*, 2023. ePrint 2022/1735.
- [31] Rate-Limiting Nullifier project. RLN documentation, 2023.
- [32] Samuel Schlesinger and Jonathan Katz. Anonymous credit tokens (ACT). IETF Internet-Draft draft-schlesinger-cfrg-act, 2025.
- [33] Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers. arXiv:1802.07344; NDSS 2019, 2018.
- [34] Erkan Tairi, Pedro Moreno-Sanchez, and Matteo Maffei. A2L: Anonymous atomic locks for scalability in payment channel hubs. In *IEEE Symposium on Security and Privacy (S&P)*, 2021. ePrint 2019/589.
- [35] The VCV-io developers. VCV-io: Verified cryptography in Lean via oracle computations. ePrint 2026/899, 2026.
- [36] Verified zkEVM project. ArkLib: Verified building blocks for SNARKs in Lean, 2026.
- [37] Cathie Yun, Christopher A. Wood, and Armando Faz-Hernández. Anonymous rate-limited credentials (ARC). IETF Internet-Draft draft-ietf-privacypass-arc-crypto, 2025.